

PyStormTracker: A High-Performance Cyclone Tracker in Python

Albert M. W. Yau ¹¶ and Kevin I. Hodges ²

¹ School of Marine and Atmospheric Sciences, Stony Brook University, Stony Brook, New York, USA ² Department of Meteorology, University of Reading, Reading, United Kingdom ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Cyclone tracking converts gridded atmospheric data into trajectories of individual weather systems. These trajectories are used to study where cyclones form, how they move and intensify, how storm tracks vary between datasets, and how circulation changes relate to weather impacts. **PyStormTracker** (PST) is an open-source Python package for cyclone detection, tracking, trajectory comparison, and storm-track analysis.

PyStormTracker provides three tracking approaches through a common interface. SimpleTracker follows local extrema, HodgesTracker implements the feature-tracking methodology developed by Hodges ([Hodges, 1994, 1995, 1999](#)), and HealpixTracker adapts feature tracking to an equal-area spherical grid ([Górski et al., 2005](#)). All three return the same Tracks representation, so downstream comparison and storm-track statistics do not depend on which tracker produced the trajectories. The package accepts labeled atmospheric data or files, records the tracked variable and preprocessing choices, and supports parallel execution.

Statement of need

Objective cyclone climatologies depend on both the tracking algorithm and the field supplied to it. The IMILAST intercomparison found substantial differences among extratropical-cyclone climatologies produced by different detection and tracking methods ([Neu et al., 2013](#)). Even within one method, the diagnostic field and spatial filtering can change the identified population. For example, filtered lower-tropospheric relative vorticity emphasizes synoptic-scale circulation differently from sea-level pressure ([Hoskins & Hodges, 2002](#)), and cyclone-change estimates can depend on whether a large-scale pressure background is removed ([Chang, 2014](#)).

Reproducible storm-track analysis therefore requires the tracked field, filtering, feature-point definition, trajectory construction, treatment of missing points, and post-tracking filters to be specified together. PyStormTracker addresses this requirement within the Python scientific ecosystem. Tracking configuration and preprocessing are retained with the output, while trajectory comparison and Eulerian and Lagrangian statistics use a common data model. The package includes cyclone and track frequency, amplitude-based statistics, Accumulated Cyclone Activity (ACA), Accumulated Track Activity (ATA), and the CORMAX and cross-validated EOF-CCA methods used to relate storm-track variability to weather impacts ([Yau & Chang, 2020](#)).

The target users are atmospheric scientists analysing reanalyses, climate-model simulations, numerical-weather-prediction output, and other gridded atmospheric datasets. The computational requirement increases as analyses move from coarse global products to higher-resolution data and non-regular spherical grids.

State of the field

TRACK implements the Hodges feature-tracking methodology and has been used extensively for extratropical storm-track analysis and reanalysis comparison (Hodges et al., 2003; Hoskins & Hodges, 2002, 2005). PyStormTracker does not treat TRACK as scientific ground truth: the published Hodges papers define the methods, while TRACK 1.5.4 provides an implementation baseline for source-parity tests. HodgesTracker reproduces the same methodological sequence—feature detection and refinement, trajectory construction, and trajectory optimization—within a Python/xarray workflow and the shared PyStormTracker data model.

Other open-source frameworks address related tracking problems. TempestExtremes provides configurable pointwise feature detection and tracking on structured and unstructured climate grids (Ullrich & Zarzycki, 2017). tobac provides a Python framework for identifying, tracking, and analysing clouds and other atmospheric objects (Heikenfeld et al., 2019). PyStormTracker has a narrower storm-track focus: its distinct contribution is a Python implementation of the Hodges methodology with TRACK 1.5.4 correspondence tests, established cyclone spectral-preprocessing configurations, and common trajectory-comparison and storm-track metrics. Contributing this functionality to a broader tracking framework would still require a separate Hodges implementation, TRACK-derived parity work, and the associated storm-track analysis model.

PyStormTracker began in 2015 as a local-extrema and nearest-neighbour tracker developed through NCAR SIParCS and presented at the 2016 AMS Annual Meeting (Yau et al., 2016). The simple tracker was later used as an independent sensitivity test in published storm-track impact analysis (Yau & Chang, 2020). The current package retains that lineage while adding the Hodges and HEALPix approaches under the same output and analysis interfaces.

Software design

The central data structure is an immutable, packed Tracks object containing aligned arrays for trajectory identifiers, offsets, times, coordinates, and sampled variables. Individual trajectories are views of these arrays rather than persistent Python objects for every track point. This design reduces object overhead for large trajectory collections and gives all trackers the same downstream representation for serialization, comparison, variable sampling, and statistics.

Meteorological inputs are normalized as labeled xarray arrays before tracker-specific calculations (Hoyer & Hamman, 2017). For supported rectangular Gauss–Legendre and Clenshaw–Curtis grids, PyStormTracker delegates reusable spherical-harmonic field operations to spharmgrid (Yau, 2026b). These include spectral filtering, spectral regridding, and default wind kinematics; ducc0 supplies the underlying spherical-harmonic transforms (Reinecke, 2020). These operations first entered PyStormTracker and were later extracted into spharmgrid so that the atmospheric field operations could be maintained, tested, and released independently. PyStormTracker retains cyclone-specific policy, including the diagnostic field, spectral band, reconstruction grid, spatial boundary treatment, and processing metadata. Reduced-Gaussian, HEALPix, polar, regional-DCT, and explicit-lmax vector paths use direct ducc0 machinery where the rectangular spharmgrid interface does not apply.

Preprocessing, feature detection, and refinement are independent across time steps and can execute concurrently. Trajectory optimization requires temporal context, so Hodges and HEALPix processing uses overlapping time segments that are spliced deterministically. Serial, Dask, and MPI backends preserve the same tracking configuration and canonical result definitions over the repository-tested scope. Dask supports task-based execution on a workstation or cluster, MPI provides distributed-memory execution, and native ducc0 threads can accelerate spherical transforms. The normal in-memory workflow does not require transformed intermediate files.

Research impact statement

PyStormTracker has public development history dating to 2015–2016 and is distributed through PyPI, conda-forge, containers, and a Zenodo software archive (Yau, 2026a). Its original simple tracker contributed an independent tracking sensitivity test to the evaluation of storm-track metrics against precipitation and strong-wind impacts (Yau & Chang, 2020). The current metrics modules implement the Eulerian and Lagrangian statistics, ATA, CORMAX, and cross-validated EOF–CCA analyses developed in that research program.

Repository-maintained validation compares HodgesTracker with TRACK 1.5.4 on a full year of 2024 ERA5 mean-sea-level pressure. For 1,464 six-hourly frames with T6–42 filtering and the common RSPLICE population, one-to-one trajectory F1 is 99.7% for F320-to-T42 reconstruction and 99.8% for F320-to-F320. Median matched-track separations are 4.1 m and 4.7 m, and 95th-percentile separations are 33.9 m and 37.9 m, respectively. These measurements quantify implementation correspondence for the stated experiments; they are not external validation of cyclone climatology.

The same controlled workload provides a performance comparison on the recorded 16-core/32-thread host. Three sequential PyStormTracker Dask repetitions, using four frame workers, four SHT threads, and four ducc0 native threads, gave median wall times of 16.97 s for F320-to-T42 and 38.29 s for F320-to-F320. Retained five-repeat TRACK 1.5.4 medians were 59.43 s and 1997.16 s, giving descriptive TRACK/PyStormTracker ratios of 3.50 and 52.16. TRACK was not rerun for the September 2026 refresh. The comparison applies to the recorded implementations, configuration, data, and machine and does not establish asymptotic scaling. Reproduction scripts, configurations, and retained TRACK reference material are maintained in the PyStormTracker-Validation repository.

AI usage disclosure

OpenAI GPT-5.6 models, including Luna, Terra, and Sol configurations used through ChatGPT and coding agents, assisted parts of the 2026 redevelopment. Uses included repository exploration, implementation proposals, refactoring, test scaffolding, code review, documentation editing, and manuscript drafting; the present manuscript was edited with GPT-5.6 Sol. The corresponding author defined the software scope and design decisions, selected and reviewed AI-assisted changes, and checked them against repository tests, primary literature, TRACK source comparison, and recorded parity and benchmark experiments. The authors remain responsible for the accuracy, originality, licensing, and scientific interpretation of the software and manuscript.

Acknowledgements

The original PyStormTracker project was supported through the National Center for Atmospheric Research 2015 SIParCS program and developed with Kevin Paul and John Dennis. The storm-track analysis framework implemented by the package grew from scientific collaboration with Edmund K. M. Chang. PyStormTracker also builds on the openly available TRACK source and the broader scientific Python ecosystem.

References

Chang, E. K. M. (2014). Impacts of background field removal on CMIP5 projected changes in pacific winter cyclone activity. *Journal of Geophysical Research: Atmospheres*, 119(8), 4626–4639. <https://doi.org/10.1002/2013JD020746>

- Górski, K. M., Hivon, E., Banday, A. J., Wandelt, B. D., Hansen, F. K., Reinecke, M., & Bartelmann, M. (2005). HEALPix: A framework for high-resolution discretization and fast analysis of data distributed on the sphere. *The Astrophysical Journal*, 622(2), 759–771. <https://doi.org/10.1086/427976>
- Heikenfeld, M., Marinescu, P. J., Christensen, M., Watson-Parris, D., Senf, F., Heever, S. C. van den, & Stier, P. (2019). Tobac 1.2: Towards a flexible framework for tracking and analysis of clouds in diverse datasets. *Geoscientific Model Development*, 12(11), 4551–4570. <https://doi.org/10.5194/gmd-12-4551-2019>
- Hodges, K. I. (1994). A general method for tracking analysis and its application to meteorological data. *Monthly Weather Review*, 122(11), 2573–2586. [https://doi.org/10.1175/1520-0493\(1994\)122%3C2573:AGMFTA%3E2.0.CO;2](https://doi.org/10.1175/1520-0493(1994)122%3C2573:AGMFTA%3E2.0.CO;2)
- Hodges, K. I. (1995). Feature tracking on the unit sphere. *Monthly Weather Review*, 123(12), 3458–3465. [https://doi.org/10.1175/1520-0493\(1995\)123%3C3458:FTOTUS%3E2.0.CO;2](https://doi.org/10.1175/1520-0493(1995)123%3C3458:FTOTUS%3E2.0.CO;2)
- Hodges, K. I. (1999). Adaptive constraints for feature tracking. *Monthly Weather Review*, 127(6), 1362–1373. [https://doi.org/10.1175/1520-0493\(1999\)127%3C1362:ACFFT%3E2.0.CO;2](https://doi.org/10.1175/1520-0493(1999)127%3C1362:ACFFT%3E2.0.CO;2)
- Hodges, K. I., Hoskins, B. J., Boyle, J., & Thorncroft, C. (2003). A comparison of recent reanalysis datasets using objective feature tracking: Storm tracks and tropical easterly waves. *Monthly Weather Review*, 131(9), 2012–2037. [https://doi.org/10.1175/1520-0493\(2003\)131%3C2012:ACORRD%3E2.0.CO;2](https://doi.org/10.1175/1520-0493(2003)131%3C2012:ACORRD%3E2.0.CO;2)
- Hoskins, B. J., & Hodges, K. I. (2002). New perspectives on the northern hemisphere winter storm tracks. *Journal of the Atmospheric Sciences*, 59(6), 1041–1061. [https://doi.org/10.1175/1520-0469\(2002\)059%3C1041:NPOTNH%3E2.0.CO;2](https://doi.org/10.1175/1520-0469(2002)059%3C1041:NPOTNH%3E2.0.CO;2)
- Hoskins, B. J., & Hodges, K. I. (2005). A new perspective on southern hemisphere storm tracks. *Journal of Climate*, 18(20), 4108–4129. <https://doi.org/10.1175/JCLI3570.1>
- Hoyer, S., & Hamman, J. J. (2017). Xarray: N-D labeled arrays and datasets in python. *Journal of Open Research Software*, 5(1), 10. <https://doi.org/10.5334/jors.148>
- Neu, U., Akperov, M. G., Bellenbaum, N., Benestad, R., Blender, R., Caballero, R., Coccozza, A., Dacre, H. F., Feng, Y., Fraedrich, K., Grieger, J., Gulev, S., Hanley, J., Hewson, T., Inatsu, M., Keay, K., Kew, S. F., Kindem, I., Leckebusch, G. C., ... Wernli, H. (2013). IMILAST: A community effort to intercompare extratropical cyclone detection and tracking algorithms. *Bulletin of the American Meteorological Society*, 94(4), 529–547. <https://doi.org/10.1175/BAMS-D-11-00154.1>
- Reinecke, M. (2020). *DUCC: Distinctly useful code collection*. Astrophysics Source Code Library, record ascl:2008.023. <https://ascl.net/2008.023>
- Ullrich, P. A., & Zarzycki, C. M. (2017). TempestExtremes: A framework for scale-insensitive pointwise feature tracking on unstructured grids. *Geoscientific Model Development*, 10(3), 1069–1090. <https://doi.org/10.5194/gmd-10-1069-2017>
- Yau, A. M. W. (2026a). *PyStormTracker: A high-performance cyclone tracker in python*. Zenodo. <https://doi.org/10.5281/zenodo.18764813>
- Yau, A. M. W. (2026b). *Spharmgrid (Version 0.1.3)*. Zenodo. <https://doi.org/10.5281/zenodo.22559210>
- Yau, A. M. W., & Chang, E. K. M. (2020). Finding storm track activity metrics that are highly correlated with weather impacts. Part i: Frameworks for evaluation and accumulated track activity. *Journal of Climate*, 33(23), 10169–10186. <https://doi.org/10.1175/JCLI-D-20-0393.1>

179 Yau, A. M. W., Paul, K., & Dennis, J. (2016). *PyStormTracker: A parallel object-oriented*
180 *cyclone tracker in python*. 96th American Meteorological Society Annual Meeting, Sixth
181 Symposium on Advances in Modeling and Analysis Using Python. [https://doi.org/10.](https://doi.org/10.5281/zenodo.18868625)
182 [5281/zenodo.18868625](https://doi.org/10.5281/zenodo.18868625)

DRAFT